UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Software

Aplicaciones Distribuidas (ISR-701)

FORM-TL-01

Taller de modelado de comunicación entre procesos distribuidos

Notas del taller

Escenario: 2 (plataforma de streaming OndaPlay)

Integrantes: Castro Coello Mario Sebastián
Loor Suárez Steffanny del Rocío
Quintero Rico Luis David
Rivas Piloso Dayver Yahir
Docente: Gleiston Cicerón Guerrero Ulloa, PhD
Nivel / paralelo: Séptimo nivel / B
Periodo: SPA 2026–2027
Fecha: 11 de noviembre de 2026

Repositorio: `https://github.com/reyshadow0/isr701-form-tl-01`

Índice

1. Escenario y procesos **2**
- 1.1. Qué es OndaPlay . . . 2
- 1.2. Actores y frontera del sistema . . . 2
- 1.3. Procesos mínimos . . . 2
- 1.4. Procesos agregados: GW, KFK y RMQ . . . 2
- 1.5. Por qué la solución es distribuida . . . 3
- 1.6. Operaciones del escenario . . . 3
- 1.7. Requisitos y situaciones de fallo . . . 3

2. Diagrama de comunicación entre procesos **4**
- 2.1. Componentes del sistema . . . 5
- 2.2. Notación utilizada . . . 5
- 2.3. Lectura del diagrama . . . 5

3. Interacciones síncronas: sockets y RPC **6**
- 3.1. S1: Solicitud de segmento de video (sockets TCP) . . . 6
- 3.2. S2: Verificación de licencia (gRPC) . . . 7
- 3.3. S3: Consulta de suscripción (gRPC) . . . 8
- 3.4. S4: Exploración del catálogo (gRPC) . . . 8
- 3.5. S5: Inicio de sesión (gRPC) . . . 9

4. Interacciones asíncronas y tolerancia a fallos **10**
- 4.1. A1: Envío de telemetría (Reproductor → Telemetría) . . . 10
- 4.2. A2: Eventos de visualización (Telemetría → Recomendaciones) . . . 10
- 4.3. A3: Cambio de suscripción (Facturación → Licencias) . . . 11

5. Declaración de uso de inteligencia artificial **13**

1. Escenario y procesos

1.1. Qué es OndaPlay

El caso viene del enunciado del escenario 2 (Anexo A de la rúbrica GA-TL-01). OndaPlay es un servicio de streaming ficticio, con suscripción, que combina catálogo bajo demanda y eventos en vivo de fin de semana, y que se ve desde el celular, la televisión o el navegador. El rediseño parte de tres fallas actuales: arranque lento del video en los eventos en vivo, reproducciones de usuarios sin suscripción vigente y datos de visualización que llegan tarde al equipo de recomendaciones.

1.2. Actores y frontera del sistema

El suscriptor usa el reproductor para iniciar sesión, buscar títulos, ver contenido y renovar su plan. El administrador de contenidos carga títulos, temporadas y metadatos en el catálogo. La pasarela de pagos es un sistema externo que procesa los cobros de las suscripciones.

Dentro de la frontera quedan los procesos que OndaPlay controla: los siete procesos mínimos, la puerta de enlace y dos brókers de mensajes. La pasarela de pagos queda fuera, y OndaPlay solo se comunica con ella desde el servicio de facturación. Los servidores de borde de la CDN, que el enunciado también lista como actor, se modelan dentro del sistema como el proceso VID, porque son los que entregan el video al reproductor.

1.3. Procesos mínimos

Cada proceso usa el mismo código que en el diagrama de la sección 2.

CLI Reproductor (cliente). Inicia sesión, muestra el catálogo, pide la licencia y los segmentos de video y envía la telemetría.

AUT Servicio de autenticación y licencias. Verifica que la suscripción esté vigente y entrega la licencia para reproducir.

CAT Servicio de catálogo. Guarda títulos, temporadas y metadatos.

VID Servidor de contenido o nodo de borde. Entrega los segmentos de video al reproductor.

TEL Servicio de telemetría. Recibe los eventos del reproductor: inicio, pausa, avance, errores y calidad.

REC Servicio de recomendaciones. Consume los eventos de visualización para actualizar las recomendaciones de cada usuario.

PAG Servicio de facturación y suscripciones. Registra renovaciones y cancelaciones, cobra a través de la pasarela de pagos y responde el estado de cada suscripción.

1.4. Procesos agregados: GW, KFK y RMQ

GW es la puerta de enlace (API Gateway). Recibe las peticiones del reproductor, las dirige a AUT o a CAT y devuelve la respuesta, así el reproductor no necesita conocer la dirección de cada servicio interno. También guarda en caché las consultas frecuentes del catálogo y reintenta las llamadas a AUT cuando una instancia falla (sección 3). Hay dos flujos del reproductor que no pasan por GW: los segmentos de video, que pide directamente al nodo de borde (S1), y la telemetría, que envía directamente a TEL (A1).

Para los flujos que no necesitan respuesta inmediata agregamos dos brókers, uno por tipo de mensaje. KFK (Apache Kafka) lleva los eventos de visualización de TEL a REC. En una transmisión en vivo son unos 5 000 eventos por segundo (R3), y Kafka los escribe en disco, así

que si REC se cae los eventos quedan en el bróker hasta que vuelva (F2). RMQ (RabbitMQ) lleva los avisos de PAG a AUT cuando una suscripción se renueva, se cancela o falla un pago. Son pocos mensajes, pero ninguno se puede perder: RabbitMQ los guarda en una cola hasta que AUT confirma que los recibió. Estos flujos se detallan en la sección 4.

1.5. Por qué la solución es distribuida

Los procesos corren en máquinas distintas y se comunican solo por la red, sin memoria compartida. El reproductor está en el dispositivo de cada suscriptor, los nodos de borde se ubican cerca de los espectadores, los servicios internos corren en el centro de datos de OndaPlay y la pasarela de pagos pertenece a otra empresa.

Repartir el trabajo también responde a los requisitos. Un solo servidor no atiende a 50 000 espectadores simultáneos (R1), por eso la carga se reparte entre varios nodos de borde. La disponibilidad del 99,9 % de las licencias (R6) exige varias instancias de AUT. Y como REC está separado del flujo de reproducción, una caída de las recomendaciones no detiene el video.

1.6. Operaciones del escenario

(a) *Iniciar sesión y explorar el catálogo.* CLI envía las credenciales a GW y GW las valida con AUT, que devuelve un token de sesión (S5). Las búsquedas y la consulta de un título van de CLI a GW y de GW a CAT (S4).

(b) *Solicitar la licencia y empezar a reproducir.* GW pide la licencia a AUT (S2) y AUT consulta a PAG el estado de la suscripción (S3). Con el token de licencia, CLI pide los segmentos al nodo de borde VID (S1).

(c) *Enviar la telemetría mientras se reproduce.* CLI envía a TEL un evento cada 10 segundos, sin esperar respuesta (A1).

(d) *Actualizar las recomendaciones a partir de lo visto.* TEL publica los eventos de visualización en KFK y REC los consume para recalcular las recomendaciones (A2).

(e) *Renovar la suscripción y reflejar el cambio en las licencias.* PAG cobra a través de la pasarela de pagos, actualiza el estado de la suscripción y publica el cambio en RMQ (A3). AUT lo recibe y usa el nuevo estado en las siguientes licencias.

1.7. Requisitos y situaciones de fallo

El enunciado del escenario 2 fija seis requisitos cuantificados y tres situaciones de fallo (tabla 1), y pide que el diseño cumpla R2 y R4 a la vez. La guía agrega una restricción: al menos una interacción con sockets y otra con RPC.

Tabla 1: Requisitos cuantificados y situaciones de fallo de OndaPlay

Código	Descripción
R1	En una transmisión en vivo puede haber 50 000 espectadores simultáneos.
R2	El video debe empezar a reproducirse en menos de 2 segundos desde que el usuario pulsa «reproducir», incluida la verificación de la licencia.
R3	Cada reproductor envía un evento de telemetría cada 10 segundos; en una transmisión en vivo son unos 5 000 eventos por segundo. Se acepta perder hasta el 1 % de esos eventos.
R4	Una suscripción vencida o cancelada debe impedir nuevas reproducciones en menos de 5 minutos.
R5	Las recomendaciones pueden actualizarse con hasta 15 minutos de retraso (consistencia eventual aceptable).
R6	El servicio de licencias debe tener una disponibilidad del 99,9 %, porque sin licencia no hay reproducción.
F1	El servicio de licencias no responde: ¿se bloquea a todos los usuarios o se permite reproducir durante un tiempo?
F2	El servicio de recomendaciones está caído durante dos horas: ¿se pierden los eventos de visualización de ese periodo?
F3	Un nodo de borde falla en plena transmisión en vivo: ¿qué pasa con los espectadores conectados a él?

2. Diagrama de comunicación entre procesos

El diagrama de OndaPlay muestra cómo se comunican los procesos que intervienen en el funcionamiento de esta plataforma de streaming. En él se representan los servicios que participan en las solicitudes del cliente, la verificación de licencias y suscripciones, la consulta del catálogo y el acceso al contenido audiovisual.

La Figura 1 presenta los procesos de OndaPlay, sus interacciones y los actores externos que participan en el sistema.

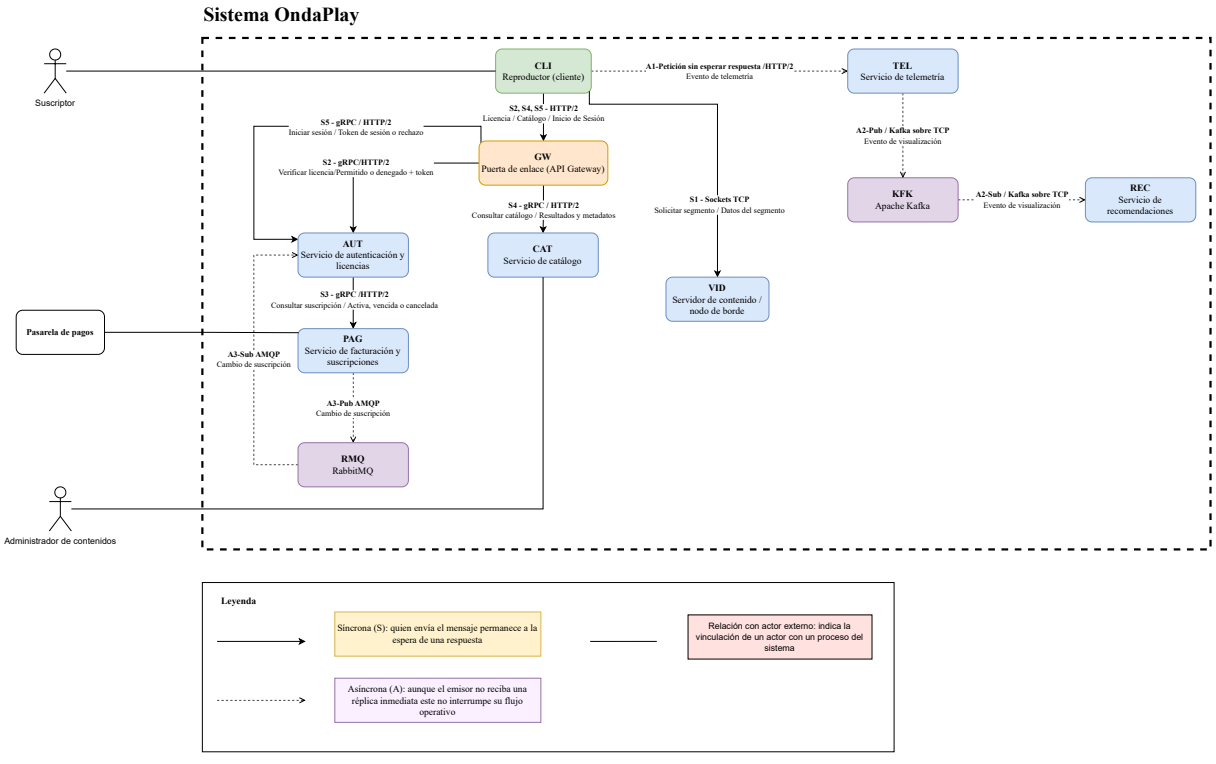


Figura 1: Comunicación entre los procesos y actores externos de OndaPlay

2.1. Componentes del sistema

Los componentes se identifican mediante códigos que permiten reconocerlos dentro del diagrama. CLI representa al reproductor (cliente), desde el cual el suscriptor accede a la plataforma, mientras que GW corresponde a la puerta de enlace que recibe y dirige las solicitudes hacia los servicios internos.

AUT se encarga de la autenticación y las licencias, CAT representa el servicio de catálogo, VID corresponde al servidor de contenido o nodo de borde, y PAG identifica al servicio de facturación y suscripciones.

También se incluyen TEL, encargado de la telemetría, y REC, representa al servicio de recomendaciones de contenido. Para la comunicación asíncrona se utilizan KFK (Apache Kafka) y RMQ (RabbitMQ), estos permiten intercambiar mensajes entre los procesos.

Fuera de la frontera del sistema se encuentran tres actores externos: Suscriptor (utiliza el reproductor), Administrador de contenidos (interactúa con el catálogo) y Pasarela de pagos (se comunica con el servicio de facturación y suscripciones).

2.2. Notación utilizada

Las flechas indican la dirección en la que se envían los mensajes entre los procesos. Se utilizan líneas continuas para representar las comunicaciones síncronas, en las que el emisor espera una respuesta antes de continuar, y líneas discontinuas para las comunicaciones asíncronas, en las que el emisor no necesita esperar una respuesta inmediata.

Cada interacción se identifica con un código. La letra S seguida de un número corresponde a una comunicación síncrona, mientras que la letra A es utilizada para una comunicación asíncrona. Junto a cada etiqueta se detallan los datos intercambiados, el mecanismo de comunicación y el protocolo utilizado.

Las relaciones con los actores externos se representan mediante líneas continuas sin puntas de flecha en ninguno de sus extremos y sin identificadores S o A (como los mencionados anteriormente). Estas líneas permiten distinguir la participación de los actores de las comunicaciones entre los procesos internos de OndaPlay

2.3. Lectura del diagrama

En las comunicaciones síncronas representadas, S1 corresponde a la solicitud de segmentos de video desde el reproductor (CLI) directamente al servidor de contenido (VID), sin pasar por la puerta de enlace (GW). Esta comunicación utiliza sockets TCP.

La interacción S2 corresponde a la comunicación entre GW y AUT para verificar las licencias. En S3, AUT consulta a PAG para comprobar el estado de la suscripción, mientras que S4 permite a GW solicitar información del catálogo a CAT. Por su parte, S5 representa el inicio de sesión. La solicitud se envía desde CLI hacia GW y luego a AUT, donde se realiza la autenticación y se devuelve un token de sesión o un rechazo. Las interacciones S2, S3, S4 y S5 utilizan gRPC sobre HTTP/2.

En las comunicaciones asíncronas, A1 representa el envío de eventos de telemetría desde CLI hacia TEL. En A2, TEL publica eventos de visualización mediante Kafka (KFK), que posteriormente son consumidos por REC para generar recomendaciones.

Finalmente, A3 representa la comunicación desde PAG hacia AUT mediante RabbitMQ (RMQ), utilizada para informar los cambios en las suscripciones. Los mecanismos y protocolos de estas interacciones asíncronas se detallan en la Sección 4.

3. Interacciones síncronas: sockets y RPC

En OndaPlay, algunas operaciones deben esperar la respuesta de otro servicio antes de continuar. Es el caso del inicio de sesión, la consulta del catálogo, la autorización de una reproducción y la entrega de segmentos de video. Aunque la comunicación sea síncrona, el reproductor puede seguir mostrando el contenido que ya tiene en su búfer mientras espera un nuevo segmento.

Se identificaron cinco interacciones. S1 utiliza sockets TCP para solicitar segmentos de video; S2, S3, S4 y S5 emplean gRPC sobre HTTP/2 para las llamadas entre servicios. Cada elección se relaciona con las necesidades de OndaPlay y con los requisitos del escenario.

3.1. S1: Solicitud de segmento de video (sockets TCP)

- **Emisor → receptor:** Reproductor (CLI) → Nodo de borde (VID). La petición llega al nodo encargado de entregar el video, sin pasar por el API Gateway.
- **Mensaje:** El reproductor envía una solicitud JSON con el identificador del contenido, el número de segmento, la calidad y el token de licencia. VID verifica el token y, si la petición es válida, devuelve la longitud del segmento seguida de sus datos binarios. Esta delimitación permite saber cuántos bytes debe recibir el cliente.
- **Tipo:** Sockets TCP.
- **Por qué es síncrona:** Para procesar el segmento solicitado, el reproductor debe esperar a que VID lo entregue. Esa espera no impide mostrar los segmentos que ya estén almacenados en el búfer.
- **Protocolo:** TCP.
- **Middleware:** No se requiere middleware de mensajería. Se plantea una conexión directa mediante sockets TCP, con un formato definido para distinguir las solicitudes y los segmentos recibidos.
- **Justificación:**
  - *Latencia:* R2 establece que la reproducción debe comenzar en menos de 2 segundos. Enviar la solicitud directamente a VID evita una llamada adicional al Gateway durante la entrega del segmento. Sin embargo, el cumplimiento de R2 también depende del tiempo de autorización, la red y la disponibilidad del primer segmento.
  - *Volumen:* R1 contempla hasta 50 000 espectadores simultáneos. Por eso, la entrega debe repartirse entre varios nodos de borde; un solo nodo no sería una base adecuada para atender toda esa carga.
  - *Respuesta inmediata:* Se requiere una respuesta dentro del tiempo disponible antes de que se vacíe el búfer. Si el segmento tarda demasiado, el video puede detenerse.
  - *Si no responde:* Si un nodo falla (F3), el reproductor espera hasta el tiempo límite configurado e intenta solicitar el segmento a otro nodo disponible. Si tampoco lo obtiene, puede producirse una interrupción.
  - *Consistencia:* El contenido, número y versión identifican cada segmento. En una transmisión en vivo, los nodos pueden recibirlo en momentos distintos; por eso, el reproductor debe recibir la versión correspondiente a la secuencia que está reproduciendo.

- **Alternativa descartada:** HLS sobre HTTP con una CDN es una alternativa habitual para distribuir video. Aquí se proponen sockets TCP para representar de forma explícita el intercambio directo entre CLI y VID; a cambio, es necesario definir el formato de los mensajes y gestionar las conexiones y los errores.

3.2. S2: Verificación de licencia (gRPC)

- **Emisor → receptor:** Reproductor (CLI) → API Gateway (GW) → Servicio de autenticación y licencias (AUT). La llamada gRPC corresponde al tramo GW–AUT.
- **Mensaje:** GW consulta si un usuario puede reproducir un contenido determinado. AUT responde *permitido* o *denegado* y, si autoriza la solicitud, emite un token de licencia firmado con una duración de 3 minutos.
- **Tipo:** RPC.
- **Por qué es síncrona:** La autorización se necesita antes de pedir el primer segmento. Por tanto, S2 debe completarse antes de iniciar S1.
- **Protocolo:** HTTP/2.
- **Middleware:** gRPC con Protocol Buffers.
- **Justificación:**
  - *Latencia:* La verificación forma parte del tiempo de inicio exigido por R2. Se utiliza gRPC porque facilita las llamadas directas entre servicios, emplea mensajes binarios y puede reutilizar conexiones HTTP/2. Esto reduce trabajo de comunicación, aunque no asegura por sí solo el límite de 2 segundos.
  - *Volumen:* Antes de un evento en vivo pueden llegar muchas solicitudes al mismo tiempo. Se prevén varias instancias de AUT para repartir la carga asociada a R1.
  - *Respuesta inmediata:* Sí. Sin una autorización válida no se inicia una reproducción nueva.
  - *Si no responde:* Para contribuir al objetivo de disponibilidad del 99,9 % (R6), GW podrá recurrir a otras instancias de AUT y realizar reintentos limitados. Estas medidas aumentan la tolerancia a fallos, pero no garantizan por sí solas ese porcentaje.

    Si todas las instancias quedan fuera de servicio (F1), no se emitirán licencias nuevas. Un reproductor con token vigente podrá solicitar segmentos mientras VID siga aceptándolo. Cuando el token venza, la reproducción necesitará una nueva autorización; si AUT continúa caído, no podrá obtenerla.
  - *Consistencia:* AUT consulta el estado de suscripción mediante S3. Si ese estado se conserva hasta 60 segundos y el token dura 3 minutos, una licencia emitida justo antes de que caduque la información almacenada podría seguir siendo válida hasta unos 4 minutos después de una cancelación.

    Bajo esas condiciones, el diseño se mantiene dentro de los 5 minutos de R4. Para sostener ese límite, AUT no debe reutilizar datos de caché vencidos y VID debe verificar la expiración del token. El plazo describe el uso de una autorización existente; no implica que una reproducción en curso se detenga inmediatamente cuando se cancela la suscripción.

3.3. S3: Consulta de suscripción (gRPC)

- **Emisor → receptor:** Servicio de autenticación y licencias (AUT) → Servicio de suscripciones y pagos (PAG).
- **Mensaje:** AUT envía el identificador del usuario. PAG informa si su suscripción está *activa*, *vencida* o *cancelada*.
- **Tipo:** RPC.
- **Por qué es síncrona:** Si AUT no tiene un estado vigente en caché, necesita la respuesta de PAG para decidir si concede la licencia.
- **Protocolo:** HTTP/2.
- **Middleware:** gRPC con Protocol Buffers.
- **Justificación:**
  - *Latencia:* AUT mantiene el estado de la suscripción en caché durante un máximo de 60 segundos. Así evita consultar PAG en cada autorización y reduce una parte de la espera asociada a R2.
  - *Volumen:* Las solicitudes repetidas dentro de ese intervalo se atienden con el dato almacenado, lo que disminuye las llamadas a PAG cuando muchos usuarios intentan reproducir contenido.
  - *Respuesta inmediata:* La respuesta de PAG se necesita antes de emitir la licencia únicamente si no existe un dato de caché vigente.
  - *Si no responde:* AUT usa el último estado guardado solo mientras siga vigente. Si no dispone de una respuesta válida, rechaza la emisión de una licencia nueva en lugar de asumir que la suscripción continúa activa.
  - *Consistencia:* Se acepta consistencia eventual, con un retraso de caché de hasta 60 segundos. Además, cuando PAG registra una cancelación, envía un aviso asíncrono para que AUT invalide el dato guardado. Ese mecanismo se explica en la sección de interacciones asíncronas.

    El aviso ayuda a actualizar antes el estado, pero no revoca automáticamente los tokens emitidos. Por eso, el límite previsto en R4 depende tanto de la duración de la caché como de la vigencia del token, según lo explicado en S2.

3.4. S4: Exploración del catálogo (gRPC)

- **Emisor → receptor:** Reproductor (CLI) → API Gateway (GW) → Servicio de catálogo (CAT). GW consulta a CAT mediante gRPC.
- **Mensaje:** El reproductor envía una búsqueda o solicita los datos de un título. CAT devuelve los resultados y, cuando corresponda, temporadas y otros metadatos.
- **Tipo:** RPC.
- **Por qué es síncrona:** GW necesita los resultados de CAT para responder a la consulta que acaba de realizar el usuario.
- **Protocolo:** HTTP/2.
- **Middleware:** gRPC con Protocol Buffers.
- **Justificación:**

- *Latencia y volumen:* GW puede guardar durante 10 minutos los resultados de consultas frecuentes, por ejemplo, los títulos destacados durante un evento. Esto evita repetir llamadas a CAT y agiliza las búsquedas más habituales.
- *Respuesta inmediata:* La consulta debe resolverse en un tiempo razonable para que el usuario pueda elegir qué ver; no requiere la misma exigencia temporal que la entrega de un segmento de video.
- *Si no responde:* GW podrá mostrar una respuesta guardada si existe una versión utilizable. De lo contrario, informará que la búsqueda no está disponible. Mostrar datos anteriores evita dejar la pantalla vacía, aunque algunos detalles podrían estar desactualizados.
- *Consistencia:* Se tolera un retraso en la actualización de metadatos del catálogo. Los permisos de reproducción no se deciden con esta caché, sino mediante S2 y S3.

3.5. S5: Inicio de sesión (gRPC)

- **Emisor → receptor:** Reproductor (CLI) → API Gateway (GW) → Servicio de autenticación y licencias (AUT). Al igual que en S2, el tramo entre GW y AUT utiliza gRPC.
- **Mensaje:** El reproductor envía las credenciales de acceso a través de GW. AUT comprueba la identidad del usuario y devuelve un token de sesión si la validación es correcta; en caso contrario, rechaza el acceso. Este token no es el mismo que autoriza la reproducción en S2.
- **Tipo:** RPC.
- **Por qué es síncrona:** El cliente espera el resultado del inicio de sesión antes de permitir el acceso a las funciones que requieren autenticación.
- **Protocolo:** HTTP/2.
- **Middleware:** gRPC con Protocol Buffers.
- **Justificación:**
  - *Latencia:* El inicio de sesión ocurre antes de que el usuario pulse *reproducir*; por tanto, no se incluye en los 2 segundos definidos por R2 para iniciar el video.
  - *Volumen:* Muchos espectadores pueden iniciar sesión poco antes de una transmisión de R1. AUT debe poder atender ese aumento de solicitudes sin afectar las verificaciones de licencia de S2.
  - *Respuesta inmediata:* Se necesita una confirmación para habilitar el acceso, pero no hay motivo para repetir la solicitud mientras AUT siga procesándola.
  - *Si no responde:* Si AUT queda fuera de servicio (F1), no se podrán iniciar sesiones nuevas. Las sesiones que ya sean válidas podrán seguir utilizándose mientras las funciones consultadas estén disponibles. Sin embargo, al vencer una licencia de S2, su renovación también dependerá de que AUT vuelva a responder.
  - *Consistencia:* Para evitar aceptar contraseñas anteriores o cuentas bloqueadas, la validación debe consultar información de autenticación actualizada. No basta con llamar a esta operación *consistencia fuerte*: el resultado dependerá de cómo se almacenen y actualicen las credenciales y los bloqueos. S5 valida la identidad; S2 y S3 comprueban el derecho a reproducir contenido.

4. Interacciones asíncronas y tolerancia a fallos

En la plataforma de OndaPlay empleamos comunicación asíncrona para aquellos flujos donde el emisor no requiere una respuesta rápida para continuar con su flujo de trabajo, priorizando la disponibilidad y la baja latencia al usuario final. Para la gestión de mensajes utilizamos dos middlewares especializados: Apache Kafka (`KFK`) para la ingesta masiva de eventos de telemetría y recomendaciones en tiempo real debido a su arquitectura de registro distribuido en disco, y RabbitMQ (`RMQ`) como bróker de mensajería empresarial para la transmisión fiable y orientada a eventos de dominio entre los servicios de facturación y licencias.

4.1. A1: Envío de telemetría (Reproductor → Telemetría)

- **Emisor → receptor:** CLI (Reproductor en app/web/TV) → TEL (Servicio de Telemetría).
- **Mensaje:** Evento en JSON con los campos:
  - `user_id`, `session_id`, `content_id`
  - `event_type`,(play, pause, buffer_empty, quality_change)
  - `timestamp`, `metrics` (bitrate actual, fotogramas perdidos)
- **Tipo:** Cliente-Servidor (Petición/Respuesta no bloqueante).
- **Por qué es asíncrona:** La petición HTTP se dispara desde un hilo secundario en el reproductor mediante una estrategia de disparo y olvido (*fire-and-forget*). El cliente no bloquea la interfaz de usuario ni congela el buffer de video en espera de una respuesta en el servidor.
- **Protocolo:** HTTP/2 mediante peticiones REST/JSON sobre HTTPS.
- **Middleware:** No aplica a la recepción inicial; el servicio TEL recibe la petición HTTP directamente y deposita el evento en Apache Kafka (`KFK`).
- **Justificación:**
  - *Volumen:* Cumple con el requisito R3 al soportar 5 000 eventos/s generados durante transmisiones en vivo con 50 000 espectadores emitiendo un reporte cada 10 segundos.
  - *Pérdida tolerada:* Según R3, se acepta perder hasta un 1 % de los mensajes de telemetría sin que afecte a la métrica global ni interrumpa el servicio al usuario.
  - *Alternativa descartada:* Se descarto gRPC sincrono porque acoplaría el hilo principal del reproductor a la latencia de red. Y asi mismo, se descarto RabbitMQ para apache Kafka debido a que RabbitMQ sufriría mayor degradación ante ráfagas masivas en 5 000 eventos/s, mientras Kafka escribe en disco secuencialmente mediante logs distribuidos con mejor rendimiento.
  - *Si el receptor no responde:* El cliente guarda un buffer circular en memoria local de los últimos 3 a 5 eventos. Si TEL pasa más de 50 segundos caído, el reproductor empieza a descartar los paquetes mas antiguos para no saturar la RAM del dispositivo asumiendo que se superará el 1 % de pérdida pero tolerando que el video no se detenga.

4.2. A2: Eventos de visualización (Telemetría → Recomendaciones)

- **Emisor → receptor:** TEL (Servicio de Telemetría) → (`KFK`) (Apache Kafka) → REC (servicio de Recomendaciones).
- **Mensaje:** Objeto consolidado del evento visto

- `user_id`, `content_id`
- `watch_duration_seconds`
- `completion_percentage`, `timestamp`

- **Tipo:** Publicación / Suscripción (Pub/Sub).
- **Por qué es asíncrona:** El procesamiento de algoritmos de recomendación consume bastantes recursos y no es necesario actualizar el perfil del usuario en tiempo real instante a instante.
- **Protocolo:** Protocolo binario de Kafka sobre TCP.
- **Middleware:** Apache Kafka (`KFK`) (Topic de telemetría procesada).
- **Justificación:**
  - *Retraso aceptable:* Se ajusta al requisito R5, el cual permite hasta 15 minutos de retraso en la actualización del catálogo de recomendaciones (consistencia eventual).
  - *Alternativa descartada:* Se descartó RabbitMQ para este flujo debido a que Kafka administra la retención de datos en disco mediante particiones y archivos de registro (*commit logs*) configurados con un tiempo de retención de 7 días. Esto permite almacenar de forma segura las 2 horas de eventos acumulados durante la caída del servicio REC (F2) sin saturar la memoria RAM del servidor ni perder el orden de los mensajes.
  - *Si el receptor no responde:* Frente al escenario F2 (Recomendaciones caído por 2 horas), los mensajes se quedan retenidos de forma segura en las particiones del Topic de Kafka. Al volver a estar en línea, el servicio REC lee desde el último *offset* guardado y procesa el atraso sin perder ningún evento de visualización.

4.3. A3: Cambio de suscripción (Facturación → Licencias)

- **Emisor → receptor:** PAG (Servicio de Facturación) → `RMQ` (RabbitMQ) → AUT (Servicio de Licencias).
- **Mensaje:** Evento de dominio en JSON:
  - `event_id`, `user_id`
  - `event_type`,(SUBSCRIPTION_CANCELLED, RENEWED, PAYMENT_FAILED)
  - `expiration_timestamp`.
- **Tipo:** Publicación / Suscripción (Pub/Sub).
- **Por qué es asíncrona:** Evita que la pasarela de pagos o el proceso de cobros dependa del tiempo de respuesta del servicio de licencias.
- **Protocolo:** AMQP (Advanced Message Queuing Protocol).
- **Middleware:** RabbitMQ (Exchange de tipo Topic).
- **Justificación:**
  - *Tiempo máximo:* Responde al requisito R4 de revocar accesos en menos de 5 minutos. Si la notificación A3 se entrega correctamente a AUT, la memoria caché se invalida de inmediato y el acceso se bloquea en un máximo de 3 minutos (lo que tarda en expirar el token activo). En caso de no entregarse el aviso A3, la invalidación ocurre en 4 minutos debido a la expiración del TTL de la memoria caché (60 s) sumado al tiempo del token (3 min), coincidiendo con la ventana temporal del diseño síncrono S3.

- *Alternativa descartada:* Se descartó hacer peticiones síncronas periódicas desde AUT hacia PAG, ya que generaría tráfico de red innecesario y acoplamiento directo entre ambos servicios.
- *Si el receptor no responde:* Se utiliza entrega con confirmación explicita (*manual ACK*). Si AUT cae, RabbitMQ mantiene los mensajes persistidos en la cola. Tras superar el límite de reintentos, los eventos fallidos se envían a una (*Dead Letter Queue*) (DLQ) para auditarlos sin descartar notificaciones.

5. Declaración de uso de inteligencia artificial

Tabla 2: Herramientas de inteligencia artificial usadas como apoyo

Herramienta	**Versión**	**Para qué se usó**	**Cómo se verificó lo obtenido**
Claude Code, aplicación de escritorio (Mario Castro)	Claude Fable 5.2	Crear el repositorio con la plantilla LaTeX del curso, las ramas y las invitaciones. Resumir la guía y la rúbrica y armar el plan de trabajo. Proponer una lista inicial de procesos con códigos cortos. Redactar los mensajes de coordinación del grupo. Generar la carátula del SGA. Dar los comandos de Git y resolver errores. Revisar las cuatro secciones y el archivo del diagrama, y compilar el informe para buscar desbordes. Listar estos usos a partir del registro de la sesión.	Los comandos los ejecutó el integrante y el resultado se comprobó en GitHub. La lista de procesos se contrastó con el Anexo A de la rúbrica: se cambió por los siete procesos del enunciado y solo se conservaron los códigos. Cada observación de las revisiones se comprobó en el enunciado o en la sección citada antes de cambiar el texto. La compilación se repite con los comandos del README.
Claude, aplicación de escritorio, modo Cowork (Dayver Rivas)	Claude Opus 5.5	Explicar la tarea, la rúbrica y conceptos de mensajería: Kafka y RabbitMQ, colas y pub/sub, ACK, DLQ, retención y offset. Armar la plantilla vacía de la sección 4. Instalar Git y dar sus comandos. Revisar el texto de la sección 4 contra la rúbrica, el Anexo A y la sección 3. Probar la compilación.	Las interacciones A1, A2 y A3 se contrastaron con R3, R4, R5 y F2 del Anexo A. Los tiempos de A3 se compararon con los de S3 en la sección 3. El archivo se compiló en el equipo del integrante.
GitHub Copilot en Visual Studio Code 1.145.3 (Dayver Rivas)	1.0.103	Sugerencias automáticas al escribir. Se aceptó una por error.	La sugerencia se borró, se desactivó la extensión y se releyó el archivo completo.
ChatGPT (OpenAI), versión web (Steffanny Loor)	GPT-6	Se utilizó ChatGPT como apoyo para elaborar el código LaTeX de la sección 2, «Diagrama de comunicación entre procesos». Las explicaciones son de autoría propia. La herramienta se empleó únicamente para organizar y dar formato al contenido en LaTeX. Ejemplo de prompt: «Ayúdame a estructurar en código LaTeX la sección 2 del informe, utilizando el contenido que ya redacté e incorporando el diagrama de comunicación de OndaPlay (comunicacion.png), sin modificar mis explicaciones».	Se compiló el código LaTeX y se revisó el PDF generado para comprobar que la sección 2 y el diagrama de comunicación se visualizaran correctamente, sin errores de formato.
ChatGPT (OpenAI) (Luis Quintero)	No informada	Partir de una base para la sección 3 analizando el problema. Revisar el texto ya escrito para detectar información faltante y comprobar su coherencia gramatical y formal. Primera versión de las flechas S1 a S4 del diagrama, que después se redibujaron a mano en draw.io.	El integrante no informó cómo lo verificó. El coordinador contrastó la sección 3 con el Anexo A de la rúbrica y con las secciones 1 y 4 antes de fusionarla.